

RISC-V RV32I Processor: From Single-Cycle to a 5-Stage Pipelined Architecture

The Domain of the Project:

VLSI Design — RTL Design & Verification

Course Name

VLSI Design – SURE Pro Ed

Team Mentor(s) : Razi Ahmed & DeviPrasad Mishra Sir

Team Member : Byanajjara Ganesha

Period of the Project: 2 Months

Declaration

The project titled “RISC-V RV32I Processor: From Single-Cycle to a 5-Stage Pipelined Architecture” has been mentored by Razi Ahmed & DeviPrasad Mishra Sir, organised by SURE Trust, for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the members of the team mentioned below have worked on it successfully and enhanced their practical knowledge in the domain.

Team Member: Byanajjara Ganesha

Mentor's Name: Razi Ahmed & DeviPrasad Mishra Sir

Prof. Radhakumari

Executive Director & Founder

SURE Trust

Table of Contents

- Executive summary
- Introduction
- Project Objectives
- Methodology & Results
- Social / Industry relevance of the project
- Learning & Reflection
- Future Scope & Conclusion

1. Executive Summary

This project involved the design, implementation, and rigorous verification of a RISC-V RV32I processor in Verilog, progressing from a single-cycle implementation to a 5-stage pipelined architecture with hazard detection, data forwarding, and branch/jump flush logic. Rather than relying on manual inspection of simulation output, a self-checking testbench methodology was built for both designs: every test program asserts hand-derived expected register and memory values and the testbench automatically reports pass or fail.

This approach directly surfaced multiple real design defects — including an incorrect load-width decoding bug and a branch-flush wiring bug — that would not have been caught by casual testing. The final pipelined processor passes a comprehensive suite of directed tests covering the full base ALU, all branch and jump instructions, memory operations, hazard handling, and data forwarding, and the project is presented publicly on GitHub and LinkedIn with documented verification evidence.

2. Introduction

Background and Context

RISC-V is an open, royalty-free instruction set architecture that has seen rapid adoption across the semiconductor industry, from academic research cores to production silicon efforts at major computing and semiconductor companies. Hands-on implementation of an RV32I core is therefore a directly relevant, portfolio-grade skill for RTL design and verification roles.

Problem Statement / Goals

The goal of this project was to design a functionally correct RV32I processor core from first principles — first as a single-cycle implementation to validate the base datapath and instruction decoding, and then as a 5-stage pipelined implementation requiring hazard detection, operand forwarding, and control-flow flush logic — and, critically, to verify both implementations with a level of rigor closer to professional verification practice than typical coursework testing.

Scope and Limitations

The project covers the RV32I base integer instruction set: arithmetic, logic, shift, and comparison operations (register and immediate forms); loads and stores at word, halfword, and byte granularity with correct sign/zero extension; all six branch types; JAL and JALR; and the upper-immediate instructions LUI and AUIPC. It does not implement the M (multiply/divide) or C (compressed) extensions, and does not currently implement CSR-based exception and trap handling — ECALL and EBREAK are decoded but do not trigger observable trap behaviour.

Innovation Component

Beyond RTL implementation, the project's distinguishing element is its verification methodology: a self-checking testbench with hand-traced expected values for every register and memory location touched by each test, and a deliberate test-design discipline — for example, using separated “guard” and “target” registers in branch and jump tests so that a broken control-flow instruction produces a detectably wrong result rather than silently converging to the same answer as a working one.

3. Project Objectives

- Implement the RV32I base instruction set: R-type and I-type ALU operations, LUI/AUIPC, all load and store variants (LB/LH/LW/LBU/LHU/SW), all six branch types, and JAL/JALR.
- Progress the design from a single-cycle datapath to a 5-stage pipelined datapath (IF, ID, EX, MEM, WB) with hazard detection, EX-EX and MEM/WB-EX forwarding, load-use stalling, and branch/jump flush logic.
- Build an automated, self-checking verification environment rather than relying on manual inspection of waveforms or console output.
- Identify and root-cause real functional defects using systematic debugging — waveform tracing, instruction-encoding decode, and variable isolation — rather than guesswork.
- Deliverables: verified RTL for both single-cycle and pipelined implementations, a self-checking test suite covering distinct instruction categories, documented bug fixes with before/after evidence, and a public GitHub repository with a LinkedIn presentation of the work.

4. Methodology & Results

Methods / Technology Used

A classic single-cycle RV32I datapath was implemented first — program counter, instruction memory, register file, ALU, immediate generation, branch comparator, data memory, and write-back multiplexer — to validate correct instruction decoding and execution. This was then extended into a 5-stage pipelined datapath by inserting IF/ID, ID/EX, EX/MEM, and MEM/WB pipeline registers and adding a hazard detection unit, a forwarding unit, and branch/jump flush logic. Verification followed a self-checking methodology throughout: for each test program, the expected final register and memory values were derived by hand from RV32I instruction semantics before running the simulation, and the testbench automatically compares actual simulation output against these expected values, reporting PASS or FAIL per assertion rather than requiring manual inspection of hex dumps.

Tools / Software Used

- Verilog for RTL design; Verilator for simulation
- GTKWave for waveform inspection
- RISC-V GNU toolchain (riscv64-unknown-elf-as, -ld, -objcopy) to assemble test programs into instruction memory images
- Bash build scripts for automated compile-and-simulate workflows

Project Architecture

The processor progressed through two architectures, summarised below.

Stage / Module	Function
PC Unit / PC-Src MUX	Selects the next instruction address — sequential, branch target, or jump target
Instruction Memory	Fetches the 32-bit instruction word at the current PC
Decoder	Generates control signals (ALU operation, register write enable, memory access type, immediate format, load size/sign) from the opcode, funct3, and funct7 fields
Register File	32 × 32-bit general-purpose registers; x0 hardwired to zero
ALU / Branch Comparator	Executes arithmetic and logic operations, and evaluates branch conditions
Data Memory / Load Unit / Store Unit	Handles word, halfword, and byte-granularity memory access with correct sign/zero extension
Hazard Detection Unit (pipeline only)	Detects load-use hazards and stalls the pipeline for exactly one cycle
Forwarding Unit (pipeline only)	Forwards ALU results from the EX/MEM and MEM/WB pipeline stages to avoid unnecessary stalls
Flush logic (pipeline only)	Squashes incorrectly-fetched instructions following a taken branch or jump

Verification Results

Both the single-cycle and pipelined implementations were verified using directed test programs, each checked automatically against hand-derived expected values.

Test Program	Result
ALU operations (R-type and I-type)	PASS
Byte / halfword / word loads and stores, with sign and zero extension	PASS
Data forwarding (EX-EX and MEM/WB-EX chains)	PASS
Load-use hazard stalling	PASS
All six branch types, taken and not taken	PASS
JAL / JALR with call-return pattern	PASS
x0 write-protection across multiple instruction types	PASS
Combined end-to-end integration test	PASS

Defects Found and Fixed

The self-checking methodology directly surfaced the following real defects during development:

- Register file read-during-write hazard: the register file lacked a write-first bypass, causing SUB and OR to intermittently return stale operand values depending on instruction timing.
- Branch/jump flush logic: a control signal was incorrectly wired back to its own output rather than to the decoder's branch-condition signal, meaning branches were never actually evaluated as taken during pipeline execution despite appearing structurally complete.
- Load-width decoding: the decoder never set the load-size and sign-extension control signals based on the instruction's funct3 field, so every load silently behaved like a signed byte load

regardless of whether it was LB, LH, or LW — caught using a diagnostic test value with its high bit set.

- A toolchain/environment migration issue (64-bit vs. 32-bit linker emulation mismatch) that initially appeared to break every test simultaneously was correctly isolated as a build-environment problem rather than an RTL defect, using instruction-encoding decode of the compiled program to prove the fetched instructions themselves were corrupted.

Final Project Working Screenshots

The following waveform captures, taken in GTKWave, show representative signals from four of the verification test programs.

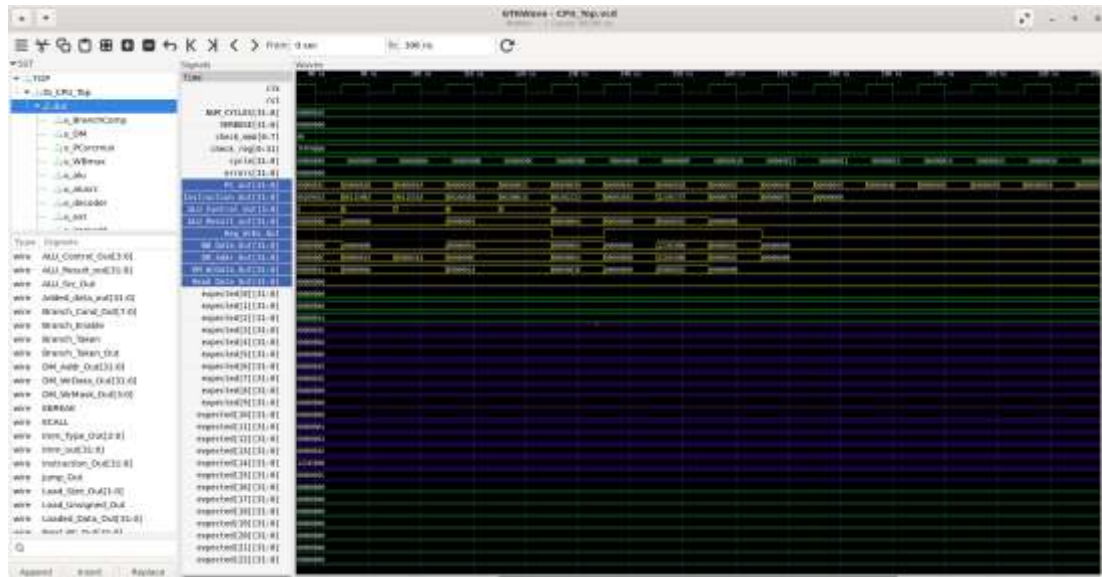


Figure 1: ALU test — R-type shift/compare sequence and the LUI/AUIPC/store-load sequence, showing correct ALU control decoding and write-back values.

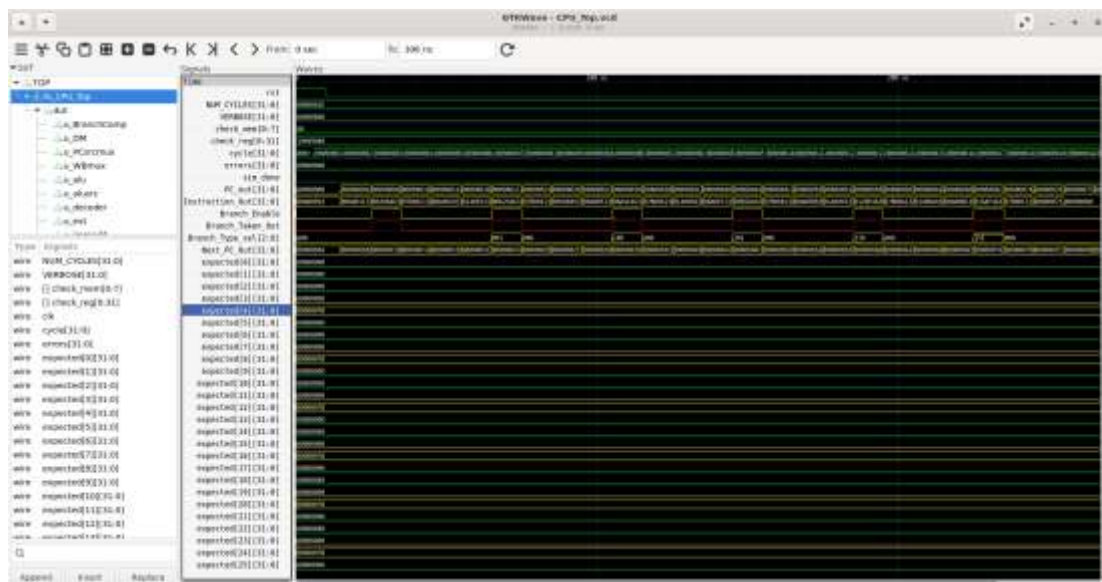


Figure 2: Branch test — Branch_Type_sel steps through all six branch encodings (BEQ–BGEU) and Branch_Taken_Out pulses correctly at each one.

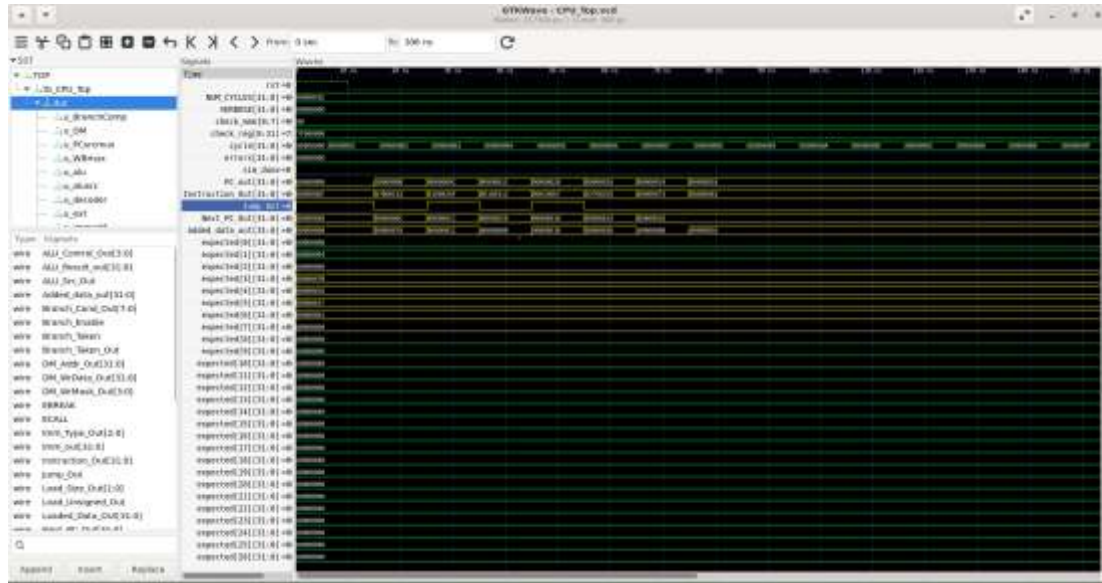


Figure 3: Jump test — *Jump_Out* pulses at each JAL/JALR instruction and *Next_PC_Out* shows the redirect away from sequential PC+4.

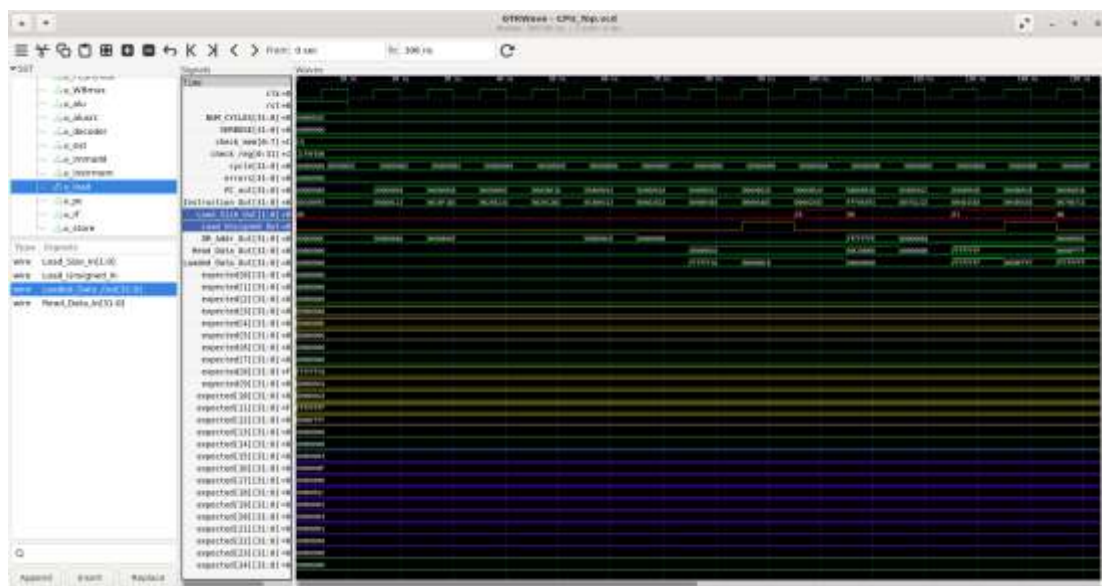


Figure 4: Load-width coverage test — *Load_Size_Out* and *Load_Unsigned_Out* change per instruction, and *Loaded_Data_Out* shows correctly sign/zero-extended results for each load variant — the direct evidence for the load-width bug fix.

Project GitHub Link

<https://github.com/Ganesha-Byanajjara8123/build-a-riscv-processor>

5. Social / Industry Relevance of the Project

RISC-V's open, royalty-free ISA has driven rapid industry adoption, from academic and hobbyist cores to production silicon efforts at major semiconductor and computing companies, making RTL design and verification skill on this ISA directly transferable to real industry roles. The verification discipline built during this project — self-checking testbenches, systematic root-cause debugging, and a clear distinction between “it compiled” and “it is actually correct” — mirrors practices used by professional RTL design and Design Verification (DV) teams, and is a meaningful differentiator for students from non-metro and first-generation engineering backgrounds seeking roles at startups and larger semiconductor companies alike.

6. Learning & Reflection

New Learnings

- Designing and integrating a complete RV32I datapath from individually verified modules.
- The mechanics and failure modes of pipeline hazards — load-use hazards, forwarding priority, and control hazards requiring flush logic — not just in theory, but by finding and fixing real instances where the flush and forwarding logic were subtly wrong.
- That a testbench's correctness matters as much as the design's: encountered and fixed a test design that could not distinguish a working branch from a broken one, and a runtime check that incorrectly flagged legal RISC-V behaviour as an error.
- Systematic root-cause isolation — distinguishing a stale build, a wrong file, a toolchain/environment mismatch, and a genuine RTL bug from one another using targeted evidence such as byte-level instruction decoding and waveform tracing, rather than guessing.

Overall Experience

The project's standard shifted over time from “get it to run” to “prove it is correct,” which was a more demanding and considerably more valuable bar to meet. It produced concrete verification evidence and a debugging narrative — including specific, defensible bug reports — that can be discussed in detail in technical interviews, rather than a project that merely “works” without evidence of how thoroughly that was established.

7. Conclusion and Future Scope

Recap of Objectives and Achievements

A complete RV32I processor was implemented and verified across two architectures — single-cycle and 5-stage pipelined — with a self-checking test suite covering the full base ALU, all load/store variants, all branch types, JAL/JALR, hazard detection, and data forwarding. Multiple real hardware defects were found and fixed using systematic debugging rather than trial and error, and the project is documented and presented publicly on GitHub and LinkedIn.

Future Scope

- Integrate the RISCOF architectural compliance framework and/or lockstep co-simulation against a golden reference model (e.g. Spike) for stronger, industry-standard verification.
- Add CSR-based exception and trap handling (illegal instruction, misaligned access, ECALL/EBREAK).
- Extend the pipeline with dynamic branch prediction (Gshare / Branch Target Buffer) and a cache subsystem with an AXI4 interface.
- Run the design through an open-source ASIC flow (OpenLane, SkyWater 130nm) to obtain real timing, area, and power figures.